

Day 5 - Searching, Binary Search & Advanced OOPS

Question + Explanation + Step-by-Step Solution + Java Programs

Topics Covered: Binary Search, Lower/Upper Bound, Rotated Sorted Array, Binary Search on Answer, Factory Pattern, Strategy Pattern, Inheritance + Interface

1. Binary Search Variant

Difficulty: Easy	Topic: Searching	Pattern: Classic Binary Search	Language: Java
------------------	------------------	--------------------------------	----------------

Question

A company stores employee IDs in sorted order. Write a Java program to search whether a given employee ID is present or not using Binary Search.

Explanation

Binary Search works only on sorted data. Instead of checking every element, it checks the middle element. If the key is smaller, it searches the left half. If the key is larger, it searches the right half. This reduces the search area by half each time.

Approach / Logic

- Array must be sorted.
- Use low and high pointers.
- Calculate $\text{mid} = \text{low} + (\text{high} - \text{low}) / 2$ to avoid overflow.
- Compare $\text{arr}[\text{mid}]$ with key.
- Move low or high based on comparison.

Steps to Solve

1. Start with $\text{low} = 0$ and $\text{high} = n - 1$.
2. Find middle index.
3. If middle value equals key, print found.
4. If key is greater than middle value, search right side.
5. If key is smaller, search left side.
6. Repeat until low becomes greater than high.

Java Solution

```
public class BinarySearchVariant {  
  
    // Method to search key using binary search  
    public static int binarySearch(int[] arr, int key) {  
        int low = 0;  
        int high = arr.length - 1;  
  
        while (low <= high) {  
            int mid = low + (high - low) / 2;  
  
            if (arr[mid] == key) {  
                return mid; // key found  
            } else if (key > arr[mid]) {
```

```

        low = mid + 1; // search right side
    } else {
        high = mid - 1; // search left side
    }
}

return -1; // key not found
}

public static void main(String[] args) {
    int[] employeeIds = {101, 105, 110, 120, 130, 150};
    int searchId = 120;

    int index = binarySearch(employeeIds, searchId);

    if (index != -1) {
        System.out.println("Employee ID found at index: " + index);
    } else {
        System.out.println("Employee ID not found");
    }
}
}

```

Expected Output

Employee ID found at index: 3

Time & Space Complexity

- Time Complexity: $O(\log n)$
- Space Complexity: $O(1)$

2. Product Inventory Range Query

Difficulty: Medium	Topic: Binary Search	Pattern: Lower Bound / Upper Bound	Language: Java
--------------------	----------------------	------------------------------------	----------------

Question

An e-commerce company stores product prices in sorted order. Write a Java program to print all products whose price is between minPrice and maxPrice.

Explanation

Lower Bound gives the first index where value is greater than or equal to minPrice. Upper Bound gives the first index where value is greater than maxPrice. All values between these two indexes are in the required range.

Approach / Logic

- Use sorted prices.
- Find lower bound of minPrice.
- Find upper bound of maxPrice.
- Print prices from lowerBound to upperBound - 1.

Steps to Solve

7. Store prices in sorted array.
8. Find first index with value $\geq$ minPrice.
9. Find first index with value $>$ maxPrice.
10. Loop through that range and print prices.

Java Solution

```
public class ProductInventoryRangeQuery {  
  
    // First index where arr[index] >= target  
    public static int lowerBound(int[] arr, int target) {  
        int low = 0, high = arr.length;  
  
        while (low < high) {  
            int mid = low + (high - low) / 2;  
  
            if (arr[mid] < target) {  
                low = mid + 1;  
            } else {  
                high = mid;  
            }  
        }  
        return low;  
    }  
  
    // First index where arr[index] > target  
    public static int upperBound(int[] arr, int target) {  
        int low = 0, high = arr.length;  
  
        while (low < high) {  
            int mid = low + (high - low) / 2;
```

```

        if (arr[mid] <= target) {
            low = mid + 1;
        } else {
            high = mid;
        }
    }
    return low;
}

public static void main(String[] args) {
    int[] prices = {100, 150, 200, 250, 300, 350, 400, 500};
    int minPrice = 200;
    int maxPrice = 400;

    int start = lowerBound(prices, minPrice);
    int end = upperBound(prices, maxPrice);

    System.out.println("Products in price range:");
    for (int i = start; i < end; i++) {
        System.out.println(prices[i]);
    }
}

```

Expected Output

Products in price range:

```

200
250
300
350
400

```

Time & Space Complexity

- Time Complexity: $O(\log n + k)$, where k is number of matching prices
- Space Complexity: $O(1)$

3. Cloud Server Capacity Planner

Difficulty: Medium	Topic: Binary Search on Rotated Array	Pattern: Search in Rotated Sorted Array	Language: Java
--------------------	---------------------------------------	---	----------------

Question

A cloud team stores server capacities in sorted order, but after rotation the array becomes {40, 50, 60, 10, 20, 30}. Write a Java program to find a required capacity efficiently.

Explanation

In a rotated sorted array, one half is always sorted. We identify which half is sorted and decide whether the key lies inside that half. This keeps binary search efficient.

Approach / Logic

- Use low, high and mid.
- Check if arr[mid] is key.
- Identify whether left half or right half is sorted.
- Move search range based on key location.

Steps to Solve

11. Calculate mid.
12. If arr[mid] equals key, return index.
13. If left half is sorted, check if key lies in left half.
14. Otherwise check right half.
15. Continue until key is found or range becomes empty.

Java Solution

```
public class CloudServerCapacityPlanner {

    public static int searchRotated(int[] arr, int key) {
        int low = 0;
        int high = arr.length - 1;

        while (low <= high) {
            int mid = low + (high - low) / 2;

            if (arr[mid] == key) {
                return mid;
            }

            // Left half is sorted
            if (arr[low] <= arr[mid]) {
                if (key >= arr[low] && key < arr[mid]) {
                    high = mid - 1;
                } else {
                    low = mid + 1;
                }
            }
            // Right half is sorted
            else {
                if (key > arr[mid] && key <= arr[high]) {
                    low = mid + 1;
                } else {
                    high = mid - 1;
                }
            }
        }
    }
}
```

```

        low = mid + 1;
    } else {
        high = mid - 1;
    }
}

return -1;
}

public static void main(String[] args) {
    int[] capacities = {40, 50, 60, 10, 20, 30};
    int required = 20;

    int index = searchRotated(capacities, required);

    if (index != -1) {
        System.out.println("Capacity found at index: " + index);
    } else {
        System.out.println("Capacity not found");
    }
}
}

```

Expected Output

Capacity found at index: 4

Time & Space Complexity

- Time Complexity: $O(\log n)$
- Space Complexity: $O(1)$

4. Book Allocation Problem

Difficulty: Medium	Topic: Binary Search on Answer	Pattern: Min-Max Binary Search	Language: Java
--------------------	--------------------------------	--------------------------------	----------------

Question

A library wants to distribute books among students. Each book has pages. Books must be allocated continuously. Minimize the maximum pages assigned to any student.

Explanation

Here we do not search directly inside the array. We search for the best answer. Minimum possible answer is the largest single book. Maximum possible answer is total pages. For each mid value, check whether allocation is possible.

Approach / Logic

- Minimum limit = maximum pages in one book.
- Maximum limit = total pages.
- Use binary search between these limits.
- If allocation is possible with mid pages, try smaller answer.
- If not possible, increase the limit.

Steps to Solve

16. Find max and sum of pages.
17. Set low = max and high = sum.
18. For every mid, check how many students are required.
19. If required students $\leq$ given students, store answer and move left.
20. Else move right.

Java Solution

```
public class BookAllocationProblem {

    public static boolean canAllocate(int[] pages, int students, int limit) {
        int studentCount = 1;
        int currentPages = 0;

        for (int p : pages) {
            if (currentPages + p > limit) {
                studentCount++;
                currentPages = p;
            } else {
                currentPages += p;
            }
        }

        return studentCount <= students;
    }

    public static int findMinimumPages(int[] pages, int students) {
        if (students > pages.length) {
            return -1;
        }
    }
}
```

```

int max = 0;
int sum = 0;

for (int p : pages) {
    max = Math.max(max, p);
    sum += p;
}

int low = max;
int high = sum;
int answer = sum;

while (low <= high) {
    int mid = low + (high - low) / 2;

    if (canAllocate(pages, students, mid)) {
        answer = mid;
        high = mid - 1;
    } else {
        low = mid + 1;
    }
}

return answer;
}

public static void main(String[] args) {
    int[] pages = {10, 20, 30, 40};
    int students = 2;

    int result = findMinimumPages(pages, students);
    System.out.println("Minimum maximum pages = " + result);
}
}

```

Expected Output

```
Minimum maximum pages = 60
```

Time & Space Complexity

- Time Complexity: $O(n \log \text{sum})$
- Space Complexity: $O(1)$

5. Online Food Delivery Pricing Engine

Difficulty: Hard	Topic: Advanced OOPS	Pattern: Factory Design Pattern	Language: Java
------------------	----------------------	---------------------------------	----------------

Question

An online food delivery app supports Normal and Express delivery. Based on customer selection, create the correct delivery object and calculate delivery charge.

Explanation

Factory Pattern hides object creation logic. The main method does not create NormalDelivery or ExpressDelivery directly. It asks the factory to return the correct object.

Approach / Logic

- Create an interface DeliveryCharge.
- Create NormalDelivery and ExpressDelivery classes.
- Create DeliveryFactory class.
- Factory method returns object based on type.
- Call calculateCharge() using interface reference.

Steps to Solve

21. Define common method in interface.
22. Implement method in each delivery class.
23. Use factory to decide object creation.
24. Use returned object to calculate price.

Java Solution

```
interface DeliveryCharge {
    int calculateCharge();
}

class NormalDelivery implements DeliveryCharge {
    public int calculateCharge() {
        return 40;
    }
}

class ExpressDelivery implements DeliveryCharge {
    public int calculateCharge() {
        return 80;
    }
}

class DeliveryFactory {

    // Factory method creates object based on delivery type
    public static DeliveryCharge getDelivery(String type) {
        if (type.equalsIgnoreCase("express")) {
            return new ExpressDelivery();
        } else {
            return new NormalDelivery();
        }
    }
}
```

```
    }  
}  
  
public class OnlineFoodDeliveryPricingEngine {  
    public static void main(String[] args) {  
        String deliveryType = "express";  
  
        DeliveryCharge delivery = DeliveryFactory.getDelivery(deliveryType);  
        int charge = delivery.calculateCharge();  
  
        System.out.println("Delivery Type = " + deliveryType);  
        System.out.println("Delivery Charge = Rs." + charge);  
    }  
}
```

Expected Output

```
Delivery Type = express  
Delivery Charge = Rs.80
```

Time & Space Complexity

- Time Complexity: $O(1)$
- Space Complexity: $O(1)$

6. Payment Gateway Design

Difficulty: Hard	Topic: Advanced OOPS	Pattern: Abstraction + Interface + Strategy Pattern	Language: Java
------------------	----------------------	---	----------------

Question

An e-commerce application supports UPI, Card and Net Banking payments. Write a Java program where payment method can be changed without modifying the main business logic.

Explanation

Strategy Pattern allows selecting an algorithm or behavior at runtime. Here each payment type is a strategy. The main code depends on Payment interface, not on concrete classes.

Approach / Logic

- Create Payment interface.
- Implement UpiPayment, CardPayment and NetBankingPayment.
- Create PaymentGateway class to use any Payment object.
- Change payment strategy at runtime.

Steps to Solve

25. Define pay(amount) method.
26. Implement different payment classes.
27. Pass required payment object to gateway.
28. Gateway calls pay() without knowing exact payment type.

Java Solution

```
interface Payment {
    void pay(int amount);
}

class UpiPayment implements Payment {
    public void pay(int amount) {
        System.out.println("Paid Rs." + amount + " using UPI");
    }
}

class CardPayment implements Payment {
    public void pay(int amount) {
        System.out.println("Paid Rs." + amount + " using Card");
    }
}

class NetBankingPayment implements Payment {
    public void pay(int amount) {
        System.out.println("Paid Rs." + amount + " using Net Banking");
    }
}

class PaymentGatewayService {
    private Payment payment;
```

```
public PaymentGatewayService(Payment payment) {
    this.payment = payment;
}

public void makePayment(int amount) {
    payment.pay(amount);
}
}

public class PaymentGatewayDesign {
    public static void main(String[] args) {
        PaymentGatewayService gateway1 = new PaymentGatewayService(new UpiPayment());
        gateway1.makePayment(500);

        PaymentGatewayService gateway2 = new PaymentGatewayService(new CardPayment());
        gateway2.makePayment(1000);

        PaymentGatewayService gateway3 = new PaymentGatewayService(new NetBankingPayment());
        gateway3.makePayment(1500);
    }
}
```

Expected Output

```
Paid Rs.500 using UPI
Paid Rs.1000 using Card
Paid Rs.1500 using Net Banking
```

Time & Space Complexity

- Time Complexity: $O(1)$
- Space Complexity: $O(1)$

7. Ride Booking System Architecture

Difficulty: Hard	Topic: Advanced OOPS	Pattern: Inheritance + Interface + Strategy Pattern	Language: Java
------------------	----------------------	---	----------------

Question

A ride booking app supports BikeRide and CarRide. Each ride has pickup and drop location. Payment can be made by Cash or UPI. Design this using inheritance and interface.

Explanation

Ride is a base class containing common ride details. BikeRide and CarRide reuse that common logic through inheritance. PaymentMode is an interface because different payment types have same behavior but different implementation.

Approach / Logic

- Create PaymentMode interface.
- Create CashPayment and UpiRidePayment classes.
- Create base class Ride with pickup/drop/fare.
- Create BikeRide and CarRide subclasses.
- Book ride and make payment using interface reference.

Steps to Solve

29. Store common ride data in parent class.
30. Use child classes for specific ride types.
31. Use interface for payment flexibility.
32. Create ride object and call bookRide().
33. Create payment object and call pay().

Java Solution

```
interface PaymentMode {
    void pay(int amount);
}

class CashPayment implements PaymentMode {
    public void pay(int amount) {
        System.out.println("Paid Rs." + amount + " by Cash");
    }
}

class UpiRidePayment implements PaymentMode {
    public void pay(int amount) {
        System.out.println("Paid Rs." + amount + " by UPI");
    }
}

class Ride {
    String pickup;
    String drop;
    int fare;

    public Ride(String pickup, String drop, int fare) {
```

```

        this.pickup = pickup;
        this.drop = drop;
        this.fare = fare;
    }

    public void bookRide() {
        System.out.println("Ride booked from " + pickup + " to " + drop);
        System.out.println("Fare = Rs." + fare);
    }
}

class BikeRide extends Ride {
    public BikeRide(String pickup, String drop, int fare) {
        super(pickup, drop, fare);
    }

    public void showRideType() {
        System.out.println("Ride Type: Bike Ride");
    }
}

class CarRide extends Ride {
    public CarRide(String pickup, String drop, int fare) {
        super(pickup, drop, fare);
    }

    public void showRideType() {
        System.out.println("Ride Type: Car Ride");
    }
}

public class RideBookingSystemArchitecture {
    public static void main(String[] args) {
        BikeRide ride = new BikeRide("College", "Home", 150);
        ride.showRideType();
        ride.bookRide();

        PaymentMode payment = new CashPayment();
        payment.pay(ride.fare);
    }
}

```

Expected Output

```

Ride Type: Bike Ride
Ride booked from College to Home
Fare = Rs.150
Paid Rs.150 by Cash

```

Time & Space Complexity

- Time Complexity: O(1)
- Space Complexity: O(1)

Day 5 Quick Revision

Topic	Use	Complexity
Binary Search	Sorted array search	$O(\log n)$
Lower/Upper Bound	Find range in sorted array	$O(\log n)$
Rotated Search	Modified binary search	$O(\log n)$
Book Allocation	Binary search on answer	$O(n \log \text{sum})$
Factory Pattern	Create object based on input	$O(1)$
Strategy Pattern	Change behavior at runtime	$O(1)$
Inheritance + Interface	Reuse common logic and support flexible behavior	$O(1)$